ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

RSL Robotic Systems Lab
Prof. Dr. Marco Hutter

Master Thesis

Target Object Search using Interactive Perception

Spring Term 2023

Supervised by:
Mayank Mittal
Dr. Vaishakh Patil

Author:
Nicola Burger

Declaration of Originality

I hereby declare that the written work I have submitted entitled

Your Project Title

is original work which I alone have authored and which is written in my own words.[1]

Author

Nicola	Burger

Student supervisors

Mayank	Mittal
Vaishakh	Patil

Committee members(s)

Mayank	Mittal
Vaishakh	Patil
Marco	Hutter

Supervising lecturer

Marco	Hutter

With the signature I declare that I have been informed regarding normal academic citation rules and that I have read and understood the information on 'Citation etiquette' (`https://www.ethz.ch/content/dam/ethz/main/education/rechtliches-abschluesse/leistungskontrollen/plagiarism-citationetiquette.pdf`). The citation conventions usual to the discipline in question here have been respected.

The above written work may be tested electronically for plagiarism.

Place and date

Signature

[1] Co-authored work: The signatures of all authors are required. Each signature attests to the originality of the entire piece of written work in its final form.

Intellectual Property Agreement

The student acted under the supervision of Prof. Hutter and contributed to research of his group. Research results of students outside the scope of an employment contract with ETH Zurich belong to the students themselves. The results of the student within the present thesis shall be exploited by ETH Zurich, possibly together with results of other contributors in the same field. To facilitate and to enable a common exploitation of all combined research results, the student hereby assigns his rights to the research results to ETH Zurich. In exchange, the student shall be treated like an employee of ETH Zurich with respect to any income generated due to the research results.
This agreement regulates the rights to the created research results.

1. Intellectual Property Rights

1. The student assigns his/her rights to the research results, including inventions and works protected by copyright, but not including his moral rights ("Urheberpersönlichkeitsrechte"), to ETH Zurich. Herewith, he cedes, in particular, all rights for commercial exploitations of research results to ETH Zurich. He is doing this voluntarily and with full awareness, in order to facilitate the commercial exploitation of the created Research Results. The student's moral rights ("Urheberpersönlichkeitsrechte") shall not be affected by this assignment.
2. In exchange, the student will be compensated by ETH Zurich in the case of income through the commercial exploitation of research results. Compensation will be made as if the student was an employee of ETH Zurich and according to the guidelines "Richtlinien für die wirtschaftliche Verwertung von Forschungsergebnissen der ETH Zürich".
3. The student agrees to keep all research results confidential. This obligation to confidentiality shall persist until he or she is informed by ETH Zurich that the intellectual property rights to the research results have been protected through patent applications or other adequate measures or that no protection is sought, but not longer than 12 months after the collaborator has signed this agreement.
4. If a patent application is filed for an invention based on the research results, the student will duly provide all necessary signatures. He/she also agrees to be available whenever his aid is necessary in the course of the patent application process, e.g. to respond to questions of patent examiners or the like.

2. Settlement of Disagreements

Should disagreements arise out between the parties, the parties will make an effort to settle them between them in good faith. In case of failure of these agreements, Swiss Law shall be applied and the Courts of Zurich shall have exclusive jurisdiction.

Place and date	Signature

Contents

Preface

This thesis presents an investigation into Target Object Search using Interactive Perception (TOSIP), a novel approach to object retrieval in cluttered environments. The research was conducted at the Robotic Systems Lab (RSL) at ETH Zurich under the supervision of Mayank Mittal and Dr. Vaishakh Patil.

I would like to express my deepest gratitude to my supervisors Mayank Mittal and Dr. Vaishakh Patil. Your unwavering support and guidance throughout the entirety of the thesis have made this a truly amazing journey for me academically as well as personally. From moments when I felt completely lost in the project to helping me overcome key challenges, you were always there to motivate and support me. I can not express enough how fortunate I am to have you as my mentors. Thank you!

I would like to extend my sincere thanks to Dr. Fan Shi for supervising the project in its early stages and contributing invaluable insights to the problem definition and throughout the research phase. Furthermore, I also had the great pleasure of working with Kaixian Qu, who was particularly helpful in analyzing and improving the open vocab capabilities of TOSIP.

Finally, I would like to acknowledge the assistance of my fellow LEO C15 buddies Emre Elbir, Ramón Calvo, Chenyu Yang, and Tianxu An. Thanks for helping me overcome technical challenges, discussing various ideas, and having loads of fun together all at the same time.

Nicola Burger, October 2023

Abstract

This thesis introduces Target Object Search using Interactive Perception (TOSIP), a method for goal-oriented object retrieval in cluttered environments within a fixed workspace. Given a language prompt and perceptual inputs, the goal of TOSIP is to perform the minimal amount of grasping or pushing steps to obtain a user-defined target object. Utilizing a 4-DOF robotic arm and a fixed camera, TOSIP employs pushing and grasping actions to manipulate objects. The methodology incorporates open vocabulary methods by combining features from said methods, specifically CLIPSeg [1], with regular features, which we refer to as Open Vocabulary Fusion. The subsequent features are used by a Double Q-Learning [2] algorithm which learns pushing and grasping synergies to retrieve objects. The presented approach outperforms its predecessor, VPG [3], in decluttering scenes more efficiently (+3.0% action efficiency) and with a higher grasp success rate of 95.1% (+17.9%). Building on top of its grasping and pushing capabilities, we show that adding Open Vocabulary Fusion enables TOSIP to retrieve target objects through descriptive prompting. We demonstrate that utilizing open vocabulary features performs better than open vocabulary predictions due to the retention of information. As a further contribution, we enhanced the VPG environment [3] by replicating and optimizing it within NVIDIA Omniverse, achieving a 17-fold increase in simulation speed.

Symbols

Symbols

I_e	image embedding
T	text embedding
F	fusion embedding
W, H	image width, height
C	channels
$\mathbb{A}$	action space
$\mathbb{D}$	orthogonally projected depth image
$\mathbb{M}$	binary difference mask
Q_ϕ	q-function equipped with ϕ weights
R	reward function
pos_c	camera position
c	change variable
c_{min}	minimum change threshold
δ	bellman error
ϕ	q-function weight parameters
γ	discount factor
α	learning rate
r_{grasp}	grasp reward
r_{push}	push reward
s_{grasps}	number of successful grasps
n_{grasps}	number of grasp actions
$n_{actions}$	number of actions
$n_{objects}$	number of objects within scene

Indices

x, y, z	x, y, z position
ψ	discretized yaw angle (rotation about z-axis)
t	action type

Acronyms and Abbreviations

AE	Action Efficiency
CNN	Convolutional Neural Network
DDQN	Double Deep Q-Network
DOF	Degree of Freedom
ETH	Eidgenössische Technische Hochschule
GSR	Grasp Success Rate
GT	Ground Truth
IP	Interactive Perception
LLM	Large Language Model
ML	Machine Learning
OV	Open Vocab
POC	Proof of Concept
RGB	Red, Green, Blue
RL	Reinforcement Learning
RSL	Robotic Systems Lab
SOTA	State-of-the-Art
TOS	Target Object Search
TOSIP	Target Object Search using Interactive Perception

Chapter 1

Introduction

1.1 Target Object Search

We refer to Target Object Search (TOS) as the process of a robotic system searching for a user-specified object within an environment. It describes a smart and thorough search for an object, just like that of a person looking for something. This task encircles a wide array of practical applications in robotics, such as:

1. **Dangerous Tasks**
   Replacing humans with robots to perform dangerous tasks like searching for ores in mines.
2. **Search & Rescue**
   Provide additional support in time-sensitive rescue operations in the event of e.g. earthquakes, landslides, or avalanches.
3. **Exploration**
   Exploring environments inaccessible to humans, e.g. space or deep sea exploration.
4. **Object Retrieval**
   Perform various object manipulation tasks with common household objects.

Given the complex nature of the aforementioned tasks, we opt to focus on the simplest use case, object retrieval of common household objects on a tabletop. We

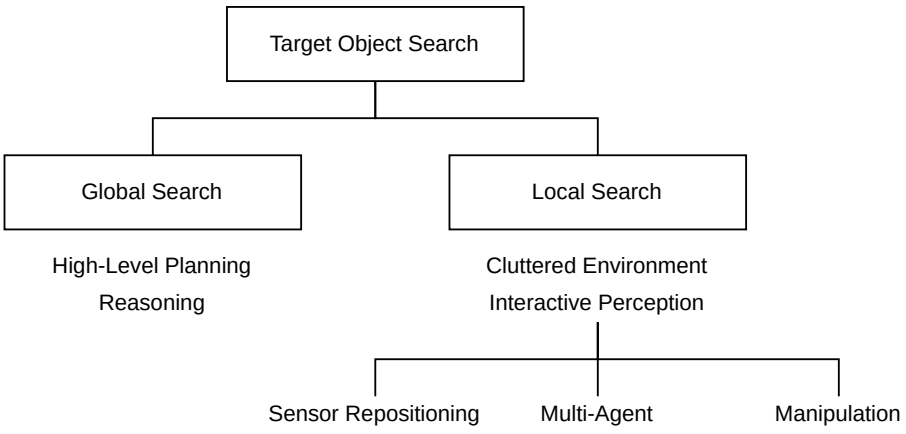
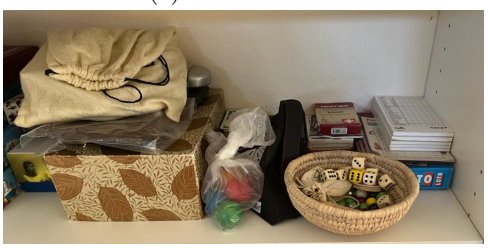


Figure 1.1: Categorized Overview of Target Object Search

(a) Dice occluded



(b) Dice visible after IP

Figure 1.2: Local Search using Manipulation (find the dice)

strive to provide a functional proof of concept (POC) that can be extrapolated to enable more challenging tasks in the future. Having chosen a use case, we additionally split the task of TOS into two subproblems, global and local search (see Fig. 1.1).

Global Search

Global search describes the act of finding areas in an environment where the target object could most likely be. Effectively generating a probability map of the chances to encounter the target object at specific locations in an environment. This requires high-level planning, as well as reasoning capabilities. High-level planning is required to track previous searches within the environment and update the probabilities given new observations. Further, this planning should integrate a cost-benefit analysis, where the next step should be the optimum with respect to the cost of navigating to a new location and the probability of encountering the target there. The reasoning capabilities should provide an intuition on where the target could likely be. For example, a spoon is most likely to be found in a kitchen drawer, a dishwasher, or on a dining table. Thanks to the advances in large language models (LLM), such as GPT-4 [4], we can get good predictions on where to look for objects (see appendix A.1). Combining these predictions with open vocab capabilities and environment mappings with a method that encapsulates both, like LERF [5], Hydra [6], or OpenScene [7], could provide a simple and effective global search method. As a sidenote, *open vocab* methods, in the context of machine learning and computer vision, refers to the capability of a system to understand and process a wide and potentially unlimited range of words or phrases.

Local Search

In contrast to global search, local search limits itself to a smaller, potentially cluttered environment and consists of using Interactive Perception (IP) to find objects which could be occluded. As an example, Fig. 1.2 illustrates the problem of finding an occluded object within a constrained workspace. In this example, the target object is a die. However, the object is not visible at first (see Fig. 1.2a). To find

occluded objects, it is required to use IP. There are three types of IP: Sensor Repositioning, Multi-Agent, and Manipulation.

1. **Sensor Repositioning**
   Repositioning the sensors can reveal new objects from a different viewpoint.
2. **Multi-Agent**
   Using multiple agents allows for multiple perspectives to sense the environment.
3. **Manipulation**
   Allows robots to interact with the environment, enabling occluded objects to be viewed.

These methods help track down target objects. However, sensor repositioning and multi-agent methods do not allow for completely occluded objects to be seen. Therefore, it is of utmost importance to integrate manipulation into local search. This is further illustrated in the example discussed previously (see Fig. 1.2b), where manipulating the bag on top of the bowl allows the dice to be seen.

1.2 Problem Statement

Summarizing the previously discussed thematics, this thesis focuses on local search using manipulation in cluttered environments. Additionally, the task is limited to the retrieval of everyday household objects and shall integrate open vocab capabilities to define the target object using natural language prompting. Retrieving the target should furthermore be accomplished in as little steps as possible by optimizing for the minimal number of required actions. This task leaves us with the following goals:

1. Enable open vocab object description using natural language prompting.
2. Optimize for minimal required actions between pre-programmed primitives: pushing and grasping.

And finally, the assumptions we make for the task at hand (see Fig. 1.3):

(A) Fixed workspace.

(B) Fixed camera with known extrinsics.

(C) 4 degree of freedom task space: x, y, z, ψ.

(D) Prehensile and non-prehensile actions (pushing and grasping).

1.3 Contributions

This thesis presents four significant contributions to the field of robotics, particularly in the domain of interactive perception in and open vocabulary grounding. The key contributions are outlined as follows:

1. **Simulation Environment Enhancement**
   A technical contribution is the replication and optimization of the VPG environment [3] within NVIDIA Omniverse. This enhancement not only ensures compatibility with advanced simulation platforms but also results in a substantial increase in simulation speed, by a factor of 17.

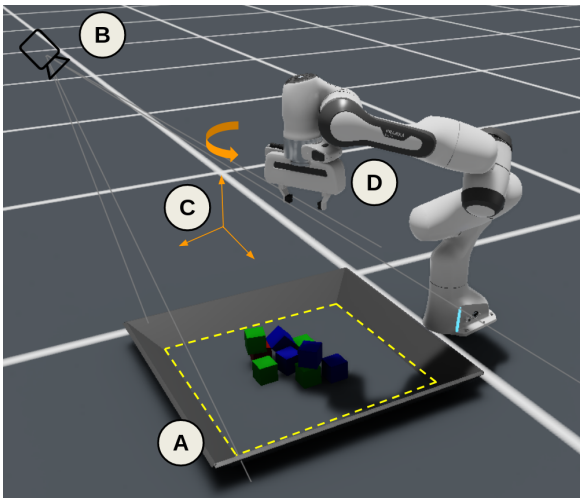


Figure 1.3: Task Space Assumptions (A through D) visualized

2. **Empirical Performance Gains**
   Our approach demonstrates a notable improvement over the previous state-of-the-art, VPG [3]. TOSIP shows a 3.0 % increase in action efficiency and a significant enhancement in grasp success rate, achieving 95.1 % (17.9 % over VPG).

3. **Integration of Open Vocabulary Methods**
   A pivotal contribution of this work is the incorporation of open vocabulary methods into the TOSIP framework. By fusing features from CLIPSeg [1] with regular features, a process we term Open Vocabulary Fusion, TOSIP gains the ability to understand and process descriptive language prompts, enhancing its object retrieval capabilities.

4. **Open Vocabulary Fusion Efficacy**
   We empirically validate that the use of open vocabulary features in TOSIP leads to better performance compared to solely relying on open vocabulary predictions. This finding underscores the importance of retaining rich information in the feature extraction process.

Chapter 2

State of the Art

Given the goals we set (see section 1.2), there are two types of methods that were analyzed. Open vocab (OV) methods were evaluated in order to enable natural language object descriptions, while object retrieval methods were assessed to find a suitable algorithm for a baseline model to expand on.

2.1 Open Vocab Segmentation Methods

Open Vocab Segmentation Methods complete the task of image segmentation using a user-specific language prompt, effectively highlighting areas which represent the given prompt. The language grounding is achieved using multi-modal embeddings, which are fused with different techniques that are discussed in the following.

2.1.1 Model Architectures

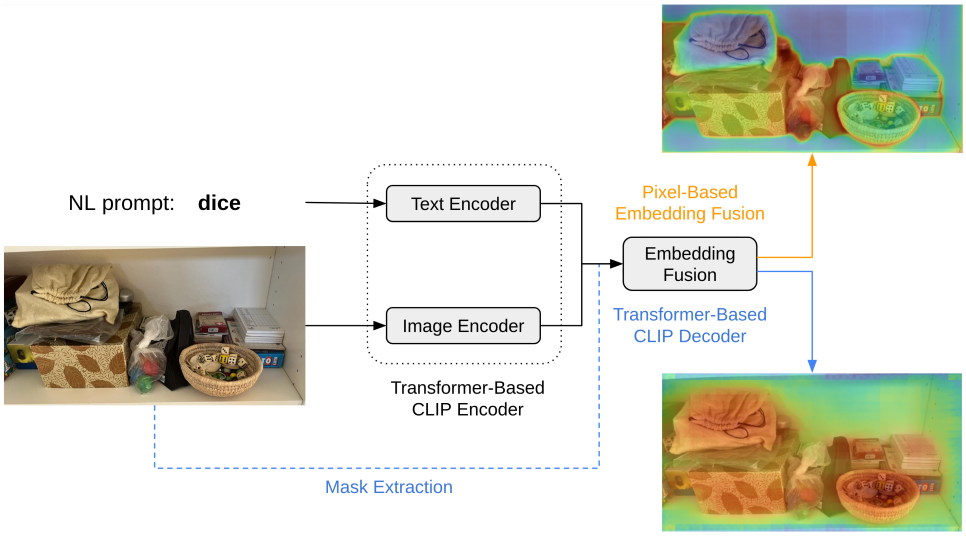


Figure 2.1: Open Vocab Segmentation Architecture

Most OV segmentation methods use multi-model embeddings (language-image pairs) and a decoding step in which these embeddings are fused to generate predictions. Although each of the discussed methods is unique, they can be classified into pixel-based, decoder-based or masked-decoder-based segmentation methods (see Fig. 2.1).

They also differ in the used multi-modal embedding generation methods, which are discussed in the following.

Multi-Modal Embeddings

A vast majority of OV segmentation methods extract multi-model embeddings through either CLIP [8] or ALIGN [9]. In essence, the two methods are trained in a similar manner using image-caption pairs, the only major difference being the datasets. On the one hand, CLIP is trained on less, but higher quality data, whilst ALIGN is trained on a larger but lower quality dataset. Nevertheless, both models provide similar results, and even though ALIGN marginally outperforms CLIP on common benchmarks [10, 11], the latter is more prevalent in most grounding tasks.

Pixel-Based Segmentation

Pixel-based segmentation methods compare pixel embeddings with text embeddings on a per-pixel basis, followed by a decoder [12, 13]. The text encoder remains frozen throughout the training process while the image encoder and decoder are trained. LSeg [12] demonstrates this approach well through their architecture (see Fig. 2.2). The text embeddings T are correlated with each pixel embedding $I_{e,ij}$ by computing the respective dot products (see eq. (2.1)). In this case, a large array of N different prompts are used. For our case, we would only require the target object description prompt ($N = 1$). While there are suitable pixel-based segmentation methods such as LSeg [12] or DenseCLIP [13], the required dot product over each pixel results in slow inference times. For instance, LSeg was chosen as a viable OV segmentation method for the pipeline, however, with a slow inference time of 3.72 s it was not selected as a candidate method for the problem at hand.

$$F_{ijk} = I_{e,ij} \cdot T_k \tag{2.1}$$

where: F : fused embeddings $\in \mathbb{R}^{HxWxN}$
I_e : image embeddings $\in \mathbb{R}^{HxWxC}$
T : text embeddings $\in \mathbb{R}^{NxC}$

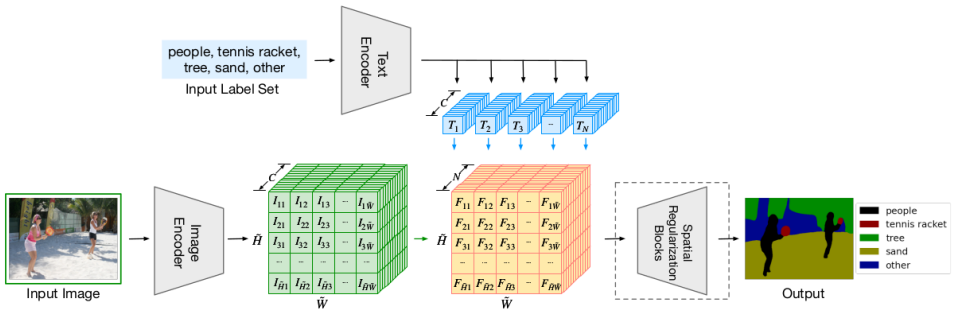


Figure 2.2: LSeg Architecture

Decoder-Based Segmentation

In decoder-based OV segmentation methods, only the decoder is trained, while the image and text encoders are frozen. The multi-modal embeddings are fused

together, which can be accomplished in various implementation-specific ways. Notable examples of such methods are Fusioner [14] (using cross-modality fusion), and CLIPSeg [1] (using a combination of FiLM [15] and linear interpolation). CLIPSeg showed promising and reliable results at a fast inference time of $330ms$. Furthermore, it features scalable input sizes, which is why this method was chosen as the OV segmentation method to be implemented into the TOSIP pipeline.

Masked-Decoder-Based Segmentation

More recently, decoder-based OV segmentation methods have been extended by adding masks as priors to the decoder, achieving more accurate predictions. Notable methods are OpenSeg [16], OVSeg [17], and SAM (Segment Anything Model) [18]. SAM in particular has gained a lot of attention by demonstrating high accuracy and allowing for different stages of segmentation resolutions. At the time of implementing TOSIP, SAM was not yet fully released, which is why it was not integrated. As a future step, one could replace the CLIPSeg embeddings with SAM embeddings.

Evaluation

Out of the available OV segmentation methods analyzed, CLIPSeg provided the fastest inferences, while qualitatively performing well, if not best, on data collected from our environment. Given the scope of the project, we neglected a deep analysis on the performance of existing methods, as we reasoned that we can replace a good model for a better one in a future step. Furthermore, the CLIPSeg latent space $\mathbb{R}^{64\times18\times18}$ provided a good starting point for our pipeline, which will be discussed later (see chapter 3). It is important to note that the chosen embeddings/predictions can be switched with other OV segmentation methods.

2.2 Object Retrieval Methods

We classify methods that use interactive perception (IP) to retrieve objects as object retrieval methods. This definition allows for a wide array of different tasks as well as a broad range of implementations. Therefore, we limit ourselves to methods that focus on either finding target objects or decluttering scenes through the use of at least one grasping action. These methods can be split into three categories: hand-crafted, vision-language-action, and reinforcement learning methods.

2.2.1 Hand-Crafted Methods

We classify methods as hand-crafted if they use explicit representations with specialized modules for each part of their pipeline. Effectively, they are not end-to-end methods and use a collection of state-of-the-art (SOTA) techniques to accomplish their respective tasks. Some reoccurring and often implemented modules consist of: object detection, object blocking relationships, declutter graph, visual grounding, and pose estimation modules. For instance, in ACTD [19], a declutter graph determines which object occludes most of the scene, which is then either pushed or grasped, depending on the various manipulation criteria. Another example is shown with INVIGORATE [20], where a user defines the target object, and a robotic gripper manipulates the scene until the user confirms the target to have been grasped successfully. The aforementioned tasks result in a diverse set of subtasks, which in the case of INVIGORATE is solved through the use of various submodules, such

as object detection, object blocking relationships, visual grounding and pose estimation. Furthermore, hand-crafted methods are not robust to changes in the task space. In contrast, end-to-end methods can be trained to generalize better.

2.2.2 Vision-Language-Action Methods

Due to the advances in LLM's, new opportunities have arisen in the form of reasoning. These capabilities have previously been limited to natural language processing. However, more recent vision-language-action methods have been developed, which integrate sensing and robotic actions. This is achieved by expanding an LLM's input and output with sensor signals (e.g. images, poses, etc.) and actions (in the form of robotic actuators). Notable contributions can be found in RT-2 [21], Gato [22], and SayCan [23], which are all trained for language as well as physical tasks using a robot interface. Unfortunately, these models require massive datasets and are computationally extremely expensive due to the number of weights they have to train (which often lie in the billions). Given the computational resources, this is a very promising field to pursue. A major advantage being the ability to interpret user-specific tasks in varying situations, hence, generalizing the task space to everyday situations and demands.

2.2.3 Reinforcement Learning

Reinforcement Learning (RL) is a subfield of machine learning (ML) where an agent learns to make decisions by interacting with an environment to maximize a reward function [24]. RL is adept at finding optimal policies in complex, dynamic environments and can adapt to new or changing conditions. This property allows tasks such as object retrieval to be learned through exploration and is end-to-end trainable. This is achieved through exploring different strategies in changing environments. To successfully learn a policy, RL requires a large number of samples, which are typically collected from simulations.

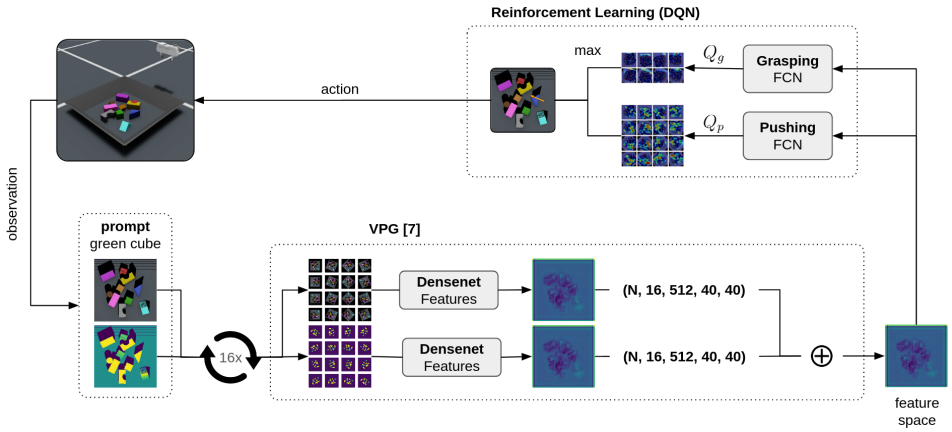


Figure 2.3: VPG Pipeline

In RL, techniques can be broadly categorized into two types: on-policy and off-policy methods. On-policy methods directly evaluate and improve the policy but are sample inefficient and highly dependent on the current policy for exploration [24]. On the other hand, off-policy methods are sample efficient and less reliant on the current policy for exploration [25]. Given the nature of our task, to achieve good OV segmentation from CLIPSeg, we require high image resolution. This limits our

sample efficiency, which is why off-policy methods are better suited for the task at hand.

An off-policy method that does an excellent job of decluttering cluttered scenes using grasping and pushing actions is *Learning Synergies between Pushing and Grasping with Self-supervised Deep Reinforcement Learning* (VPG) [3]. VPG uses Q-Learning to find the next best action in the environment. It focuses on learning synergies between pushing and grasping actions to pick up one object at a time to declutter scenes. The pipeline is visualized in Fig. 2.3. VPG uses DenseNet [26] features from orthographically projected RGB and depth image inputs, which are used for Q-Learning. It is worth mentioning that the assumptions discussed in section 1.2 overlap with the assumptions used in VPG.

Chapter 3

Method

As discussed in section 1.2, this project focuses on integrating open vocab capabilities for goal-oriented object retrieval in cluttered environments. To do so, it was chosen to build on top of a functional object retrieval method, which shall be expanded by OV capabilities. Given the discussed object retrieval methods in section 2.2, we opt to use VPG [3] due to its great decluttering competence.

By integrating OV capabilities, we ensure goal-oriented object retrieval, while retaining basic decluttering to find occluded target objects. Therefore, we expand VPG by replacing the feature generation module with what we call a Open Vocab Fusion module and further optimize the general VPG pipeline with a faster simulation environment and improved Q-Learning strategies. However, the significant contribution of this thesis lies in the Open Vocab Fusion module, which, to our current knowledge, is a novel way to integrate OV capabilities into RL methods.

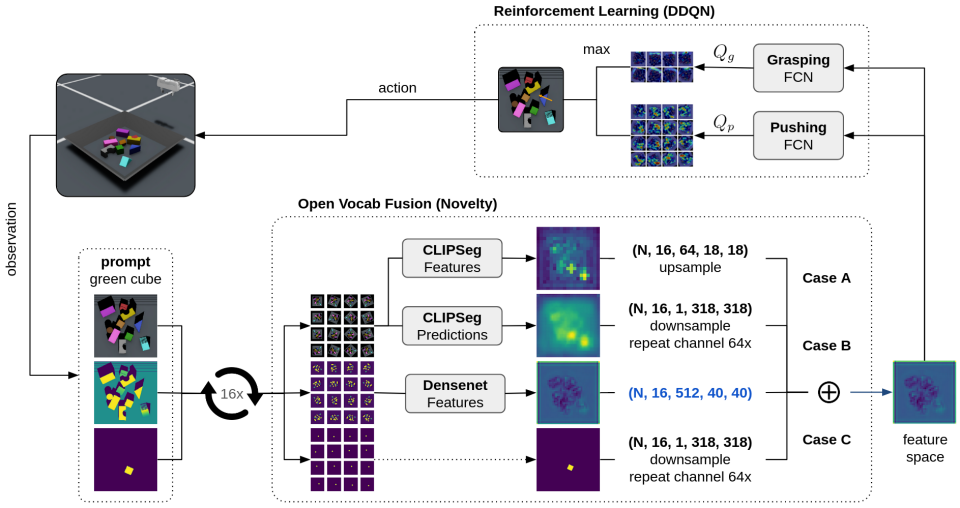


Figure 3.1: TOSIP Pipeline

As can be seen in Fig. 3.1, the pipeline consists of three basic building blocks: the environment, Open Vocab Fusion, and RL using Double Deep Q-Networks (DDQN). Actions performed in the environment result in observations, which consist of a prompt (target object description), RGB, and depth images. The observations are then preprocessed and given to the Open Vocab Fusion module, which integrates

the previously chosen CLIPSeg segmentation method (see section 2.1.1). The Open Vocab Fusion infers DenseNet and CLIPSeg features that are then concatenated into a feature space. These latents are fed to the Q-Learning module, which uses two DDQN for grasping and pushing actions respectively. The resulting logits represent Q-Values, which are used to determine the next action in the pipeline.

3.1 Environment

The performance of machine learning and reinforcement learning models is fundamentally tied to the data they are trained on. For optimal results, this data must be both high in quality and large in quantity. This necessity underscores the importance of creating an environment that is not only realistic and fast but also incorporates a degree of noise to enhance the model's generalization and robustness. Specifically, in the context of Q-Learning, we rely on a trio of elements: observations, actions, and rewards. These are essential for the successful update of the Double Deep Q-Networks (DDQNs). Furthermore, in order to obtain accurate and reliable inferences from CLIPSeg, it is crucial to have high-quality scene renderings. These renderings should come from a simulation platform that closely mimics real-world conditions. The following sections delve deeper into the individual submodules of this environment, discussing their roles and significance in the broader context of our research.

3.1.1 Simulation Platform

Typical simulation platforms for RL in robotics are Gazebo [27], MuJoCo [28], PyBullet [29], CoppeliaSim [30], and NVIDIA Omniverse. Given that VPG [3] used CoppeliaSim and already provides a viable simulation setup for the task at hand, we considered using it directly. However, upon evaluation, we identified two major limitations with CoppeliaSim: its slower performance compared to other platforms like NVIDIA Omniverse, and its lack of advanced rendering capabilities. Therefore, we decided to use the more up-to-date and faster NVIDIA Omniverse in combination with Isaac and Orbit [31], which makes use of SOTA rendering capabilities. Isaac additionally allows for parallelization of environments, which increases sample efficiency. For a comparison of CoppeliaSim and NVIDIA Omniverse, we neglect parallelization and test the scene generation and step speed of the two platforms. We find that Omniverse demonstrates superior runtimes as can be seen in table 3.1.

Table 3.1: Simulation Runtimes

Task	CoppeliaSim	Omniverse
Scene Generation $[ms]$	26'665	227
Step $[ms]$	3'573	204

- **Scene Generation**
  Randomly spawning one object after another into the workspace. After the last object is spawned, the simulation shall continue until all objects are at rest.
- **Step**
  Here, we refer to a step as a high-level action taken, which can either be a push or a grasp. The movements of the gripper are controlled by a state machine which executes subtasks for either action.

3.1.2 Scene

The scene is made up of arbitrary objects, which are spawned randomly into a predefined workspace. Both of which are further discussed here. As a reference, see Fig. 3.2 for all of the further discussed environment modules.

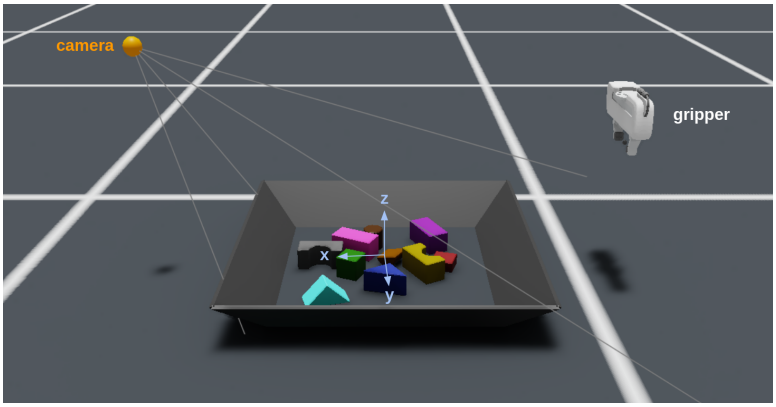


Figure 3.2: Environment Scene Setup, origin illustrated in blue

Workspace

The scene consists of an optional tray, which confines the workspace by 0.448 m in width and length. This space is chosen based on the orthographic image resolution of 224 px in width and height, which is discussed in section 3.1.3. The workspace size, in combination with the orthographic image resolution, allows each pixel to represent 2 mm of the environment workspace, see eq. (3.1).

$$\frac{w}{W_o} = \frac{l}{H_o} = 2\,\mathrm{m\,px^{-1}} \tag{3.1}$$

where: H_o : orthographic image height = 224 px
W_o : orthographic image width: 224 px
l : workspace length = 0.448 m
w : workspace width = 0.448 m

Objects

We use the same set of 7 shapes found in VPG [3], which are visualized in Fig. 3.3. These shapes were chosen because they are relatively easy to grasp. They are further colorized by applying materials and randomly spawned into the workspace.

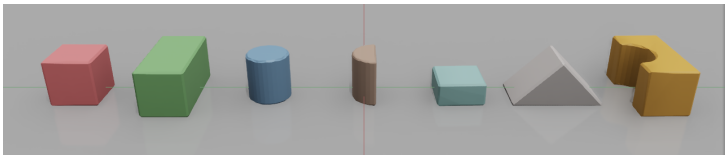


Figure 3.3: Objects Colored

Sampling occurs by spawning one object at a specified (x, y) position and fixed height $z = 0.3\,\text{m}$. A scene is generated when all objects come to a rest (see algorithm 1). We compare two methods, uniform and gaussian spawning, and evaluated them based on their aptitudes to generate occlusions.

Algorithm 1 Scene Generation

```
sim, dt, objs                          ▷ simulation, timestep, array of objects
pos ← rand_positions(len(objs))
for i in range(len(objs)) do           ▷ spawn one object at a time
    objs[i].spawn(pos[i])
    sim.step(dt)
end for
while not all objs.at_rest() do        ▷ wait until all objects come to rest
    sim.step(dt)
end while
```

- **Uniform Spawning**
  Uniform spawning results in equal probabilities of an object being spawned anywhere within the workspace boundaries.

- **Gaussian Spawning**
  Gaussian spawning uses a gaussian distribution centered at $(x, y) = (0, 0)$ with a user-defined standard deviation while also being bound by the workspace limits.

A qualitative analysis between the two spawning methods demonstrated that gaussian spawning provides a higher aptitude of occlusions. Because Q-Learning benefits from seeing more occlusions during training, gaussian spawning was elected as the method to be used.

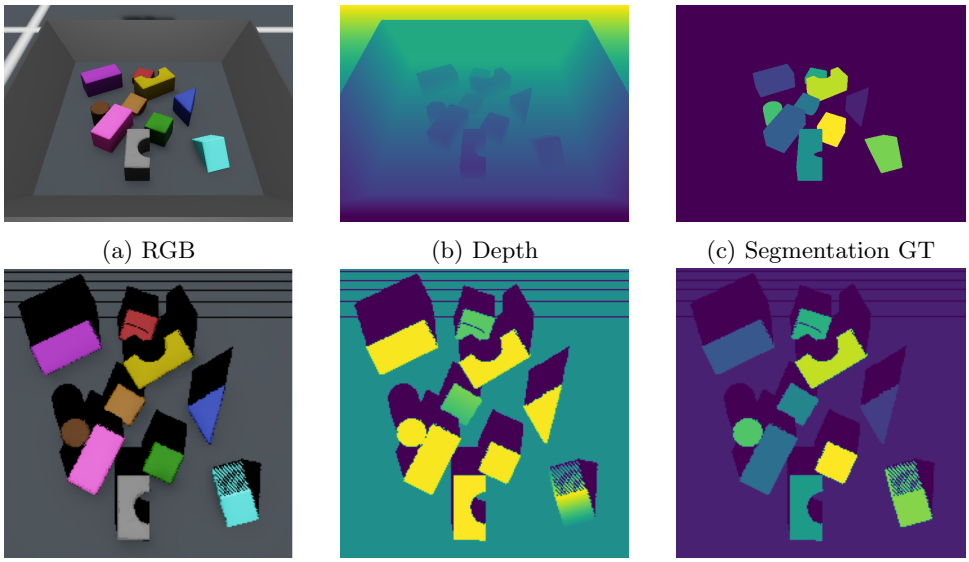
3.1.3 Observations

We equip each scene in the environment with a camera at $pos_c = (0.5\,\text{m}, 0\,\text{m}, 0.5\,\text{m})$. The camera is oriented towards the workspace, which is illustrated with rays (see Fig. 3.2). For each rollout, a scene is randomly generated upon which the camera records an image of the workspace. The camera renders RGB, depth, and segmentation ground truths, with a pixel resolution of 640×480 pixels (see Fig. 3.4a, Fig. 3.4b and Fig. 3.4c). These images are orthogonally projected along the z-axis, resulting in orthographic images with a resolution of 224×224 pixels (see Fig. 3.4d, Fig. 3.4e and Fig. 3.4f). Finally, one object within the scene is picked at random as a target object.

Let us assume the target object is the green cube, that is visualized in the scenes. Since we already have a target object elected, we no longer require the segmentation ground truths of the other objects within the scene. Therefore, we remove them from the orthographically project image, resulting in a binary mask, representing the segmentation of the target object (see Fig. 3.5). The orthographically projected images and the target object make up the observations which are provided by the environment. It is important to note, that observations only include the segmentation GT for bench-marking (case C). For TOSIP cases A and B, the observation consists of RGB, depth and target object.

Preprocessing

Because of the discretized yaw space ψ, observations are padded and rotated to represent different rotations/directions. Grasp actions require 8, while push actions require 16 discretized spaces. Taking the maximum action space ψ of the two actions, the images are rotated 16 times in a preprocessing step. To rotate the images without any loss of information, they are first padded. Images are then rotated and used as inputs for the Open Vocab Fusion module. Further information and illustrations of the preprocessing step are shown in appendix A.2.



(a) RGB (b) Depth (c) Segmentation GT

(d) RGB (e) Depth (f) Segmentation GT

Figure 3.4: Perspective Images (640 × 480 pixels) (a, b, c) and Orthographically Projected Images (224 × 224 pixels) (d, e, f)

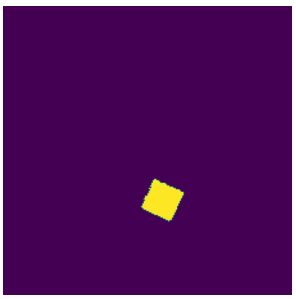


Figure 3.5: Segmentation GT Binary Mask

3.1.4 Actions

The environment is equipped with a 4 degree-of-freedom (DOF) robotic gripper, which can move along the x, y, z axis and rotate about the z axis (ψ). The actions executed in the environment can be either grasping or pushing actions (referred to as action type). The decision to discretize the positions and rotations stems from the intrinsic characteristics of Q-Learning and the specific requirements of

our application. Discretization aligns well with the tabular nature of traditional Q-Learning, where continuous action spaces pose challenges in terms of computational complexity and convergence stability. In our approach, discretizing the rotations ψ by 22.5° and the x and y positions by 2 mm matches the orthographic image resolution of the workspace, providing a balance between action space granularity and practical execution feasibility. Moreover, discretization simplifies the learning process, as it reduces the complexity of the action space, making it more manageable for our Q-Learning algorithm. Finally, the z-position is computed through addition/subtraction of the depth at the chosen x, y position. The action space is defined in eq. (3.2). It is important to note that these actions do not use closed-loop control but rather an open-loop state machine that computes the trajectory keypoints throughout an action.

$$\mathbb{A} \in \{t, \psi, x, y\} \tag{3.2}$$

where: t : action type (grasp or push) $\in \mathbb{Z}_2$
ψ : discretized yaw angle $\in \mathbb{Z}_{8/16\ (grasp/push)}$
x : x position $\in \mathbb{Z}_{224}$
y : y position $\in \mathbb{Z}_{224}$

Grasping Actions

For grasping actions the z position is computed using the orthogonally projected depth at (x, y) and subtraction of 10 mm (see eq. (3.3)). 10 mm is enough for objects to be grasped successfully, whilst not too much that it would crash into them if not accurately positioned. Since grasps are ambiguous at an offset of 180°, we discretize ψ for grasping actions by 22.5° for 180°, resulting in 8 possible actions.

$$z = \mathbb{D}(x, y) - 10\,\text{mm} \tag{3.3}$$

where: $\mathbb{D}$: orthogonally projected depth image $\in \mathbb{R}_{\geq 0}^{224 \times 224}$
z : z position $\in \mathbb{R}_{\geq 0}$

Pushing Actions

For pushing actions, the z position is computed analogously, except that 10 mm are added rather than subtracted (see eq. (3.4)). By adding an offset of 10 mm the push will not result in a crash with the ground or objects. The push is then executed along a direction of length 100 mm. In contrast to grasping actions, the push directions are not ambiguous at 180°. Therefore, ψ is discretized by 22.5° for 360°, resulting in 16 possible actions.

$$z = \mathbb{D}(x, y) + 10\,\text{mm} \tag{3.4}$$

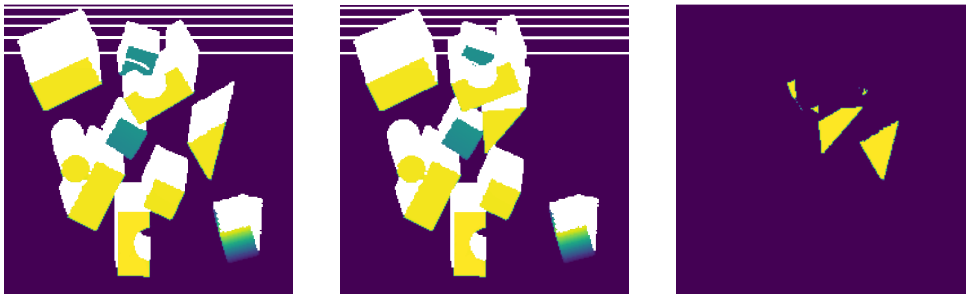
3.1.5 Rewards

Grasping and pushing actions are rewarded uniquely. For grasping actions, a grasp is defined as successful if an object is able to be lifted by the gripper. This is computed by checking if the object position is within a radius of 20 mm of the gripper position at the final step of the state machine trajectory. An unsuccessful grasp attempt is penalized with -1, while successful grasps are rewarded with $+1$ (see eq. (3.5)). Ideally, the grasped object should be the target object we are looking for. To enforce goal-oriented behavior, we reward a successful grasp of the target

object with +5.

$$r_{grasp} = \begin{cases} \text{if fail: } -1 \\ \text{if success: } +1 \\ \text{if target: } +5 \end{cases} \tag{3.5}$$

The quality of push actions is evaluated as the change that it causes within the environment. More specifically, we want to enforce pushing actions to push objects around and declutter the scene effectively. This can be achieved in a variety of methods. VPG [3] uses a simple approach, by comparing depth images before and after an action. A minimal threshold of pixel changes in the depth images is determined for a grasp to be classified as successful. Keeping it simple, this thesis replicated the same approach, proving good performances in VPG. The exact computation of the change variable is explained in the following.



(a) Depth pre-action (b) Depth post-action (c) Difference Mask $\mathbb{M}$

Figure 3.6: Difference computation

Assume a pushing action that effectively causes objects to be moved around. The depth images before and after the execution of the push action are illustrated in Fig. 3.6a and Fig. 3.6b respectively. The two images are compared by checking for changes. Shadows are ignored, and immediately excluded from being counted as a change, since they occur alongside an object that is displaced. Therefore, we compute a binary difference mask $\mathbb{M}$ using eq. (3.6), which is visualized in Fig. 3.6c. To quantify the change in the environment, a change variable c is introduced, which is the sum of all changes found in the difference mask $\mathbb{M}$ (see eq. (3.7)). A pushing action is defined as being successful if the change c is larger or equal to a minimal threshold c_{min} (see eq. (3.8)). The threshold was set to $c_{min} = 500$, roughly 1 % of all pixels in the image, given the best results after empirically testing different threshold values. A failure results in a penalty of -1, while success is rewarded with +0.5. We purposefully set push rewards lower than grasp rewards to enforce quick object retrieval, rather than constant pushes.

$$\mathbb{M}_{ij} = \begin{cases} 0, \text{ if } \mathbb{D}_{pre,ij} = \mathbb{D}_{post,ij} \text{ or } \mathbb{D}_{ij} = shadow \\ 1, \text{ otherwise} \end{cases} \tag{3.6}$$

$$c = \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \mathbb{M}_{ij} \tag{3.7}$$

$$r_{push} = \begin{cases} \text{if } c < c_{min}: & -1 \\ \text{if } c \geq c_{min}: & +0.5 \end{cases} \tag{3.8}$$

where: $\mathbb{M}$: binary difference mask $\in \mathbb{Z}_2^{HxW}$
c : change variable $\in \mathbb{R}$
c_{min} : change threshold for pushing actions $\in \mathbb{R}$

3.2 Open Vocab Fusion

The Open Vocab Fusion module is illustrated in Fig. 3.7. The extracted observations from the environment (discussed in section 3.1.3) are preprocessed and used in the Open Vocab Fusion module to produce a feature space that contains OV features from CLIPSeg or the segmentation ground truth. The feature space provides the input for Q-Learning that is discussed in section 3.3. We compare three different cases which will be used to determine the effectiveness of this method.

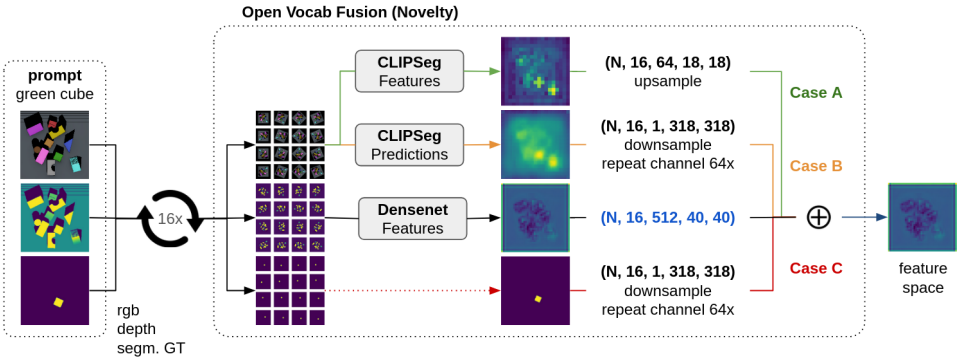


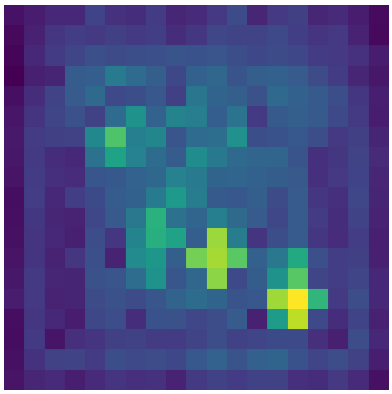
Figure 3.7: Open Vocab Fusion Overview

3.2.1 Latent Space

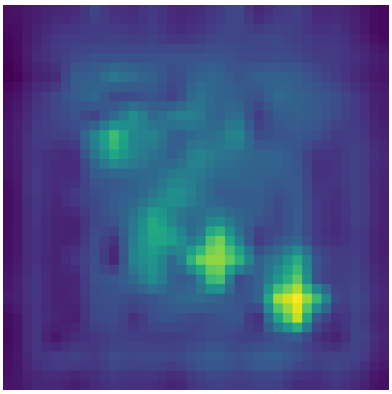
Like in the VPG [3] pipeline, we use DenseNet [26] features, which are extracted using the depth information for visual pushing and grasping tasks. However, we replace RGB DenseNet features with CLIPSeg features (case A) or CLIPSeg predictions (case B). We neglect RGB DenseNet features for simplicity, and because we did not see any noticable improvement when including them, reasoning that they are not as relevant for pushing and grasping actions. The DenseNet depth features, however, are required and their latent space (512 channels, 40 height, 40 width) is set as the fixed feature space to concatenate by.

Case A: CLIPSeg Features

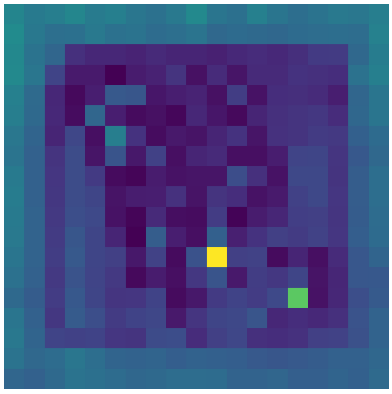
Given input images of shape (318, 318), the CLIPSeg latent space is inferred with shape (64 channels, 19 height, 19 width). This feature space must be adjusted to the DenseNet feature space. Therefore, we upsample the inferred CLIPSeg features and concatenate them as is. Since the CLIPSeg latents have multiple channels, an example is provided through mean and standard deviation over said channels in Fig. 3.8.



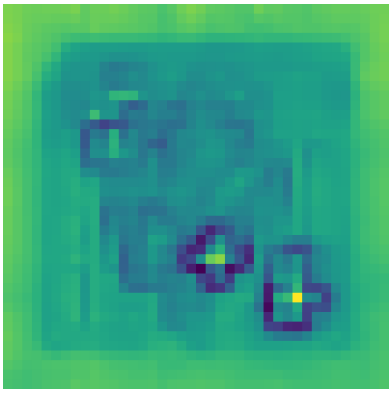
(a) Mean (64, 19, 19)



(b) Mean upsampled (64, 40, 40)



(c) Std. (64, 19, 19)



(d) Std. upsampled (64, 40, 40)

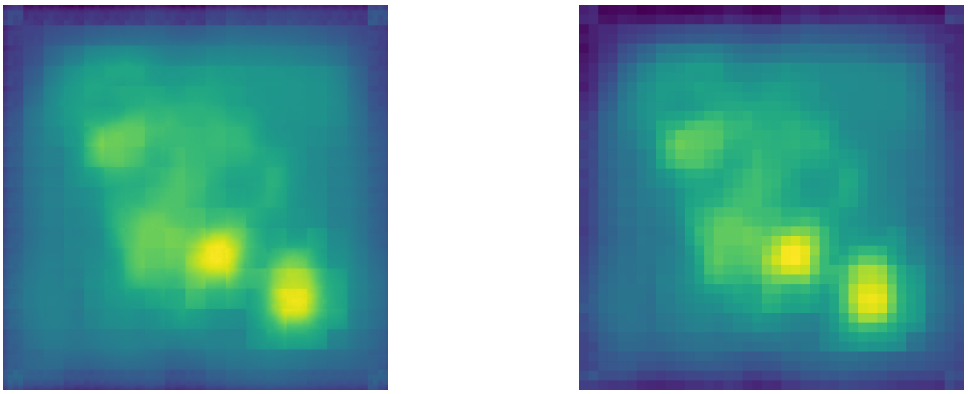
Figure 3.8: CLIPSeg Features visualized at latent space (w.r.t. Fig. A.1 and "green cube" prompt). Given that the latents have a channel size of 64, we illustrate them using their mean and standard deviation along the channel dimension.

Case B: CLIPSeg Predictions

Given input images of shape (318, 318), the CLIPSeg prediction space is inferred with shape (1 channels, 318 height, 318 width). To adjust this space to the DenseNet feature space, the predictions are bilinearly downsampled. Since the DenseNet features have a high number of 512 channels, we also repeat the CLIPSeg predictions for 64 channels. This allows the consequent convolutions in Q-Learning to better grasp the additionally provided features, rather than potentially eliminating them directly through low initialized weights. Repeating the channels further provides the benefit of comparing case B with case A, as they now have the same shape. An example of case B features is provided in Fig. 3.9.

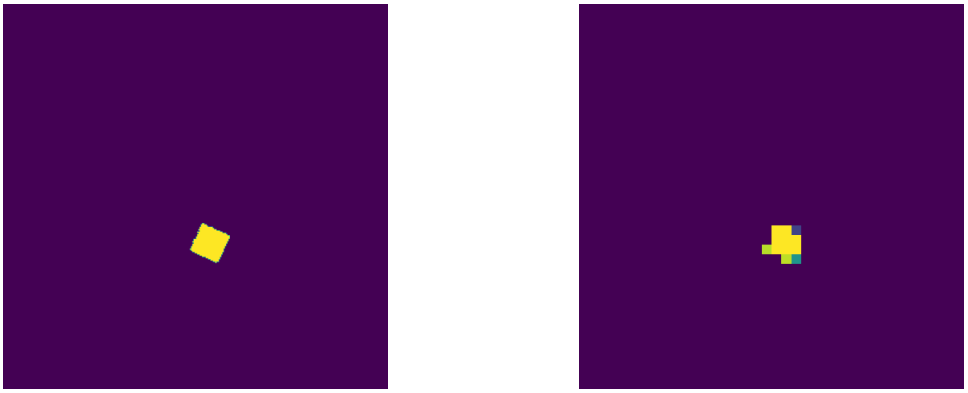
Case C: Segmentation GT

Case C acts as a benchmark for later comparison of case A and case B. By providing the segmentation ground truth, we set a baseline for the highest quality of features we could get, regardless of the OV method used. To stay consistent, we downsample and repeat these features like in case B. An illustration of segmentation ground truth features is shown in Fig. 3.10.



(a) Pred (1, 318, 318) (b) Pred downsampled (1, 40, 40)

Figure 3.9: CLIPSeg Predictions (w.r.t. Fig. A.1 and "green cube" prompt). These are the final predictions made from CLIPSeg, given the orthogonal RGB image as an input as well as the language prompt describing the target object.



(a) GT (1, 318, 318) (b) GT downsampled (1, 40, 40)

Figure 3.10: Segmentation GT (w.r.t. Fig. A.1). Absolute ground truth, which is used only for case C. This allows us to benchmark case A and B by comparing them with the optimal case C.

3.2.2 Assumptions

Comparing the three cases above, we make the following two assumptions:

1. **Information Loss**
   In case B and case C, there is an information loss happening by compressing the latent space to the DenseNet feature space through downsampling. The ratio of the remaining information is $\frac{40 \cdot 40}{318 \cdot 318} \approx 1.58\%$, which albeit being a large reduction still preserves valuable information on where the target is located at.

2. **Quality**
   Due to containing the absolute ground truth, case C has the highest OV quality. Whether case A or case B proves to have superior quality with respect to the other was not analyzed as there is no straightforward method to compare these features quantitatively. Qualitatively speaking, we believe case A to promise better OV qualities, due to it retaining information.

Given our observation, we make the following assumption on performance: case C > case A > case B. We reason that the high-quality features of case C outweigh the information loss through downsampling, therefore, outperforming case A. We further assume that case A outperforms case B due to the retained information.

3.3 Double Q-Learning

Q-Learning is a model-free RL algorithm used to find an optimal action-selection policy for a given state. It aims to learn a Q-Function $Q : S \times \mathbb{A} \rightarrow \mathbb{R}$, where S is the state space and $\mathbb{A}$ is the action space. The function $Q(\boldsymbol{s}, \boldsymbol{a})$ estimates the expected future rewards of taking action $\boldsymbol{a}$ in state $\boldsymbol{s}$. We use a form of Q-Learning called Double Q-Learning for TOSIP to learn grasping and pushing actions. By introducing OV capabilities through OV Fusion, we further enable the Double Q-Learning algorithm to learn goal-object retrieval. Due to working with a 2D feature input space, we opt to use a type of CNN as our Q-Function, which is discussed in the following section 3.3.1.

3.3.1 Q-Function

The chosen Q-Function for TOSIP is illustrated in Fig. 3.11. OV Fusion features are used as inputs, with a feature space of (576, 40, 40). The Q-Function transforms this space to (1, 318, 318), which represents the Q-Values for (x, y) actions at a certain yaw ψ for either pushing or grasping action types a. For TOSIP, a convolutional neural network (CNN) was chosen because of the two-dimensional form of the input features and due to having the applicable properties of being translationally equivariant and good at local feature extraction. The chosen CNN contains two layers, both containing a batch normalization followed by a rectified linear unit and a convolution (see table 3.2). The first convolution contains 64 kernels with a kernel size of 9. The large kernel size ensures the interpretation of object arrangements close to each other, allowing TOSIP to determine whether a push or grasp is advantageous. The second layer of the CNN should merge all of the features into one channel, which is why convolution II only contains one kernel. Because of the merging property of the second layer, a kernel size of 1 is chosen, as this layer should summarize the features rather than create new ones. Finally, the feature space of (1, 40, 40) has to be extrapolated back to the input shape of (1, 318, 318). This is achieved using bicubic upsampling. It is worth noting that both the grasping and pushing Q-Functions have the same architecture.

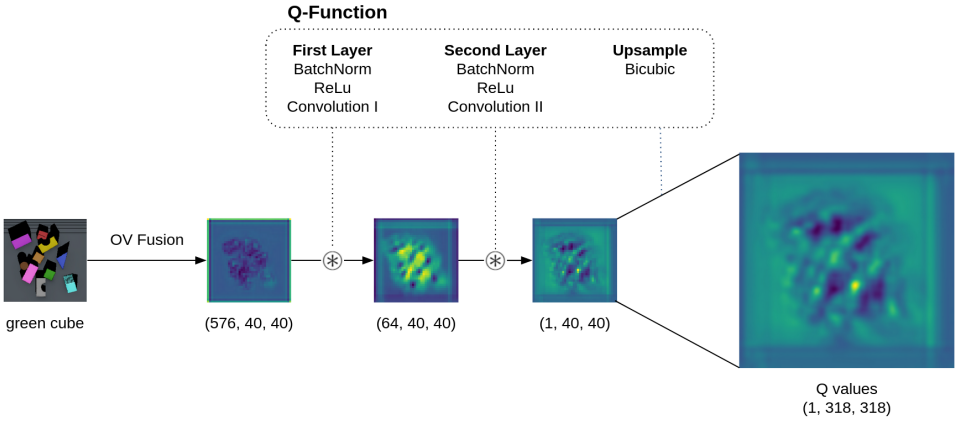


Figure 3.11: Q-Function

Table 3.2: Convolutions in Q-Function

	Kernels	Kernel Size	Stride	Bias
Convolution I	64	9	1	yes
Convolution II	1	1	1	yes

3.3.2 Algorithm

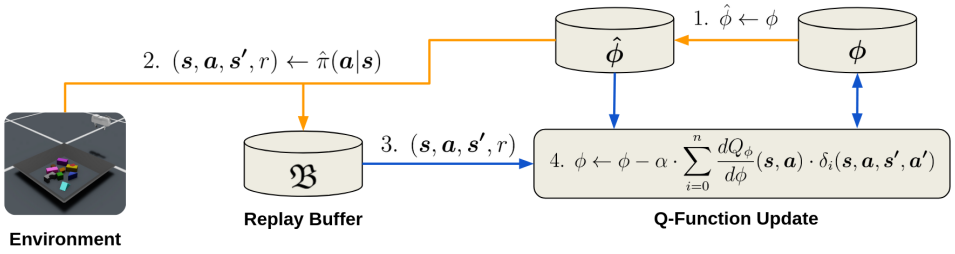


Figure 3.12: Q-Learning

Algorithm 2 Double Q-Learning

$n \leftarrow 2500$ ▷ q-learning steps
$k \leftarrow 4$ ▷ epochs per q-learning step
for i in range(n) **do**
  $\hat{\phi} \leftarrow \phi$ ▷ 1. update target network
  $\mathfrak{B} \leftarrow \hat{\pi}(\boldsymbol{a} \mid \boldsymbol{s})$ ▷ 2. collect and add data to circular buffer
  for j in range(k) **do**
    $(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s'}, r) \leftarrow \mathfrak{B}$ ▷ 3. prioritized data sampling
    $\phi \leftarrow \phi - \alpha \cdot \sum_{i=0}^{n} \frac{dQ_\phi}{d\phi}(\boldsymbol{s}, \boldsymbol{a}) \cdot \delta_i(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s'}, \boldsymbol{a'})$ ▷ 4. learning step
  end for
end for

The Double Q-Learning algorithm is summarized in algorithm 2 and illustrated with Fig. 3.12. The chosen values set for the Q-Learning steps n and epochs per Q-Learning steps k were empirically determined to give an optimal performance between runtime and learning. In a first step, we update the target Q-Function weights with those of the current Q-Function. In step 2) the data is collected from the environment using the target policy. To enforce explorations a mix of ϵ-greedy and boltzmann exploration is used. The collected data is added to the replay buffer $\mathfrak{B}$. The buffer is circular with a maximal size of 5000 samples. The collected data is then sampled in step 3) using prioritized experience replay [32], in which a minimum of one failure/success is used for grasp and push actions, respectively, allowing the Q-Functions to be trained on balanced data. Finally, the collected data is used to update the Q-Functions in step 4). To update the Q-Function, the bellman error is computed with eq. (3.9), eq. (3.10), eq. (3.11). The bellman error is used to update the current policy weights with eq. (3.12). Through hyperparameter search, we found that the best values for achieving good Q-Function learning steps are: $\gamma = 0.5$ and $\alpha = 0.01$. For further information on the algorithm itself we refer to the original publications on Q-Learning [33] and Double Q-Learning [2].

$$\delta_i = y_i - \hat{y}_i \tag{3.9}$$

$$y_i = Q_\phi(\boldsymbol{s}, \boldsymbol{a}) \tag{3.10}$$

$$\hat{y}_i = R(\boldsymbol{s}, \boldsymbol{a}) + \gamma \cdot (1 - d) \cdot Q_{\hat{\phi}}(\boldsymbol{s'}, argmax_{\boldsymbol{a'}} Q_\phi(\boldsymbol{s'}, \boldsymbol{a'})) \tag{3.11}$$

$$\phi \leftarrow \phi - \alpha \cdot \sum_{i=0}^{n} \frac{dQ_\phi}{d\phi}(\boldsymbol{s}, \boldsymbol{a}) \cdot \delta_i(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s'}, \boldsymbol{a'}) \tag{3.12}$$

where:
- δ : bellman error
- ϕ : Q-Function weight parameters
- $Q_\phi(\boldsymbol{s}, \boldsymbol{a})$: Q-Function with ϕ parameters evaluated at state $\boldsymbol{s}$ for action $\boldsymbol{a}$
- $R(\boldsymbol{s}, \boldsymbol{a})$: reward for action $\boldsymbol{a}$ in state $\boldsymbol{s}$
- γ : discount factor
- α : learning rate

Chapter 4

Results

In this chapter we compare the performance of TOSIP with and without Open Vocab Fusion. The first evaluation is based on visual pushing and grasping only, taking VPG [3] as a reference baseline to TOSIP-no-OV, discussed in section 4.1. The second evaluation is done including Open Vocab Fusion, where we compare the performance of cases A, B and C.

4.1 Visual Pushing and Grasping

To compare our methods, TOSIP, with the baseline method VPG [3], the Omniverse environment was adjusted to reflect the CoppeliaSim environment exactly. The only differences being the gripper and the simulation environment itself. Just like in VPG [3], we use the same metrics, which are:

- **Completion**
  A scene is defined as completed if all objects are removed from the workspace without failing consecutively for more than 10 unsuccessful actions.

- **Grasp Success**
  Success rate of attempted grasps over all completed rollouts (see eq. (4.1)).

- **Action Efficiency**
  Efficiency at which the scene was decluttered over all completed rollouts (see eq. (4.2)).

The test scenes used by VPG [3] were replicated in Omniverse (Fig. 4.1). These scenes reflect cases in which it is imperative to use pushing and grasping actions to declutter the scene. The results of both methods on the test scenes are shown in table 4.1. TOSIP-no-OV outperforms VPG for each metric by not only completing one test scene more but also displaying a significant increase in grasp successes. The increase in performance comes from more more training scenes seen, which is made possible by the faster NVIDIA Omniverse environment implementation, and the increased kernel size, allowing the model to capture more object arrangement relationships. The only failure to complete a scene occurs in test scene 003, where the objects are pushed out of the workspace and are effectively not reachable by the gripper any more. These results demonstrate that we are working with a viable pipeline, which we can further evaluate by including Open Vocab Fusion.

$$GSR = \frac{s_{grasps}}{n_{grasps}} \tag{4.1}$$

where: GSR : grasp success rate
s_{grasps} : number of successful grasps
n_{grasps} : number of grasp actions taken

$$AE = \frac{n_{actions}}{n_{objects}} \tag{4.2}$$

where: AE : action efficiency
$n_{actions}$: number of actions taken
$n_{objects}$: number of objects within scene

Table 4.1: Visual Pushing and Grasping

	Completion	*GSR* [%]	*AE* [%]
VPG [3]	9 / 11	77.2	60.1
TOSIP-no-OV	10 / 11	95.1	64.0

4.2 TOSIP

Demonstrating that TOSIP-no-OV performs well and learns grasping and pushing synergies, we integrate Open Vocab Fusion to enable goal-oriented object retrieval. In section 3.2.2 we made the assumption that the performances of our three cases are ranked as follows: case C > case A > case B. This assumption is verified through our evaluation, which we discuss here. Since we are no longer interested in scene decluttering, we adjust the metrics used previously while still using the grasp success rate metric *GSR*:

- **Completion**
  A scene is defined as completed if the target object is successfully grasped without failing consecutively for more than 10 unsuccessful actions.
- **Actions**
  Number of actions taken until target object is successfully grasped.

To evaluate the integrated OV capabilities, we simplify the training and test scenes. Qualitative analysis of TOSIP-no-OV showed that cubes are the easiest shapes to pick up. Hence, the object set is reduced to only cubes. Furthermore, to alleviate the problem of pushing objects outside the workspace, we add a tray to the scene, which keeps objects from crossing the boundaries. These simplifications shall highlight the OV capabilities of TOSIP because we remove unwanted failures from grasping and pushing actions which are not the focus of this analysis.

To evaluate the metrics using TOSIP, we designed a test set which is illustrated in Fig. 4.2. The test scenes are provided with a target object description prompt, which shall guide TOSIP to find objects. Scenes are terminated as soon as the target object is successfully grasped.

Given the test scenes, the following results are shown in table 4.2. They show that all scenes can be completed while also showing a 100 % *GSR*. The high completion and success rate make sense, as we simplified the environment drastically compared

to section 4.1. Furthermore, the results show that our performance assumptions are correct. Using the segmentation ground truth averages out to 1.4 actions taken until the target object is successfully grasped (with a standard deviation of 1.2). Furthermore, using CLIPSeg features leads to better performance than using CLIPSeg predictions. Finally, we concluded that OV capabilities were able to be integrated successfully while showing that it is advantageous to use the latent space rather than the predictions of CLIPSeg. We assume that the latter is also true for other types of OV methods, like for instance OVSeg [17] or SAM [18].

Even though the results confirm our assumptions, we believe that there is room for improvement with respect to the number of actions used by TOSIP. An average of 2.4 actions per scene leaves room for improvement, as most of the scenes could theoretically be completed in just one action.

Table 4.2: TOSIP Results

	Completion	*GSR* [%]	*Actions* ∅	*Actions* σ
Case A (CLIPSeg features)	100.0	100.0	2.4	1.8
Case B (CLIPSeg predictions)	100.0	100.0	3.8	3.5
Case C (Segmentation GT)	100.0	100.0	1.4	1.2

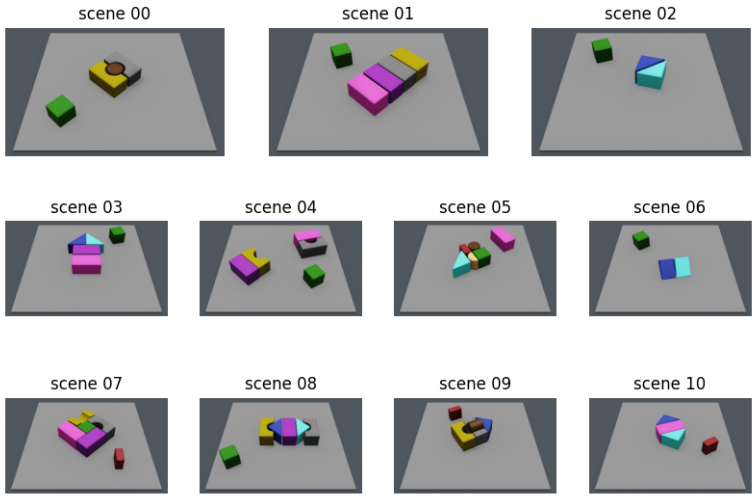


Figure 4.1: TOSIP-no-OV Test Scenes

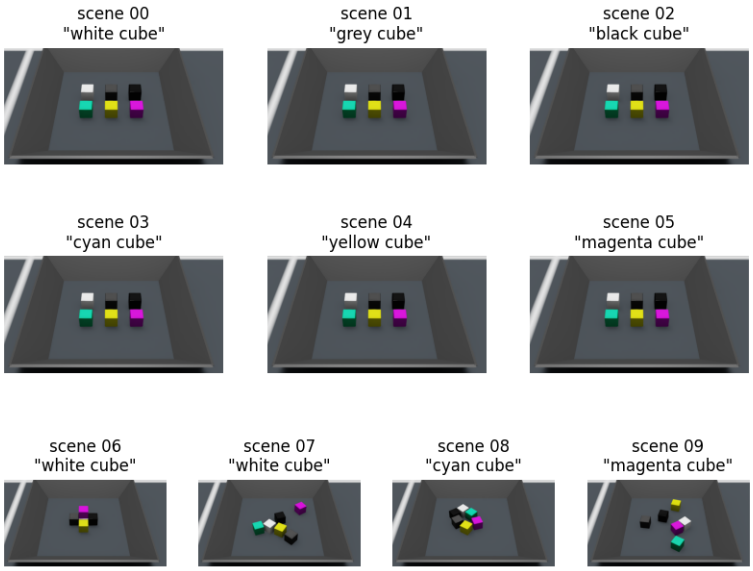


Figure 4.2: TOSIP Test Scenes (Cubes)

Chapter 5

Conclusion

Concluding this thesis, we show that integrating open vocab capabilities enables goal-oriented object retrieval in cluttered environments. By comparing different SOTA methods, we elect to use CLIPSeg for OV object descriptions and Double Q-Learning to optimize for minimal required grasping/pushing actions. A RL environment was developed in Omniverse for data collection, which is used to train Q-Functions to learn synergies between grasping and pushing actions. The Q-Functions are CNNs, which are designed to capture local object arrangements by integrating large kernels. OV capabilities were integrated into the pipeline through a module that we refer to as Open Vocab Fusion, which to the best of our knowledge presents a novel approach to incorporate OV methods into conventional feature spaces. The proposed approach outperforms its predecessor, VPG [3], in decluttering scenes more efficiently (+3.0% AE) and with a higher grasp success rate (+17.9% GSR). Building on its grasping and pushing capabilities, we show that adding Open Vocab Fusion enables TOSIP to retrieve target objects through descriptive prompting. Finally, we compare three Open Vocab Fusion cases using A) CLIPSeg features, B) CLIPSeg predictions, C) Segmentation GT, and verify our assumption that OV features outperform OV predictions due to the preservation of information.

There are 4 major improvements that we see TOSIP benefitting from in future work: 1) expanding the object set 2) hyperparameter search 3) replacing CLIPSeg with more recent OV methods and 4) sim2real. 1) Due to time constraints, we were not able to train or evaluate TOSIP on a larger set of objects. This could be achieved in a variety of ways, such as using the existing shapes used for training TOSIP-no-OV, or even expanding the object set to real scanned objects with e.g. the YCB [34] or google scanned objects [35] dataset (see Fig. 5.1). Depending on the object types, it would be possible to find the limitations on the grasping actions of the 4 DOF gripper using Double Q-Learning. 2) TOSIP could be improved by running a large-scale hyperparameter search, effectively increasing performance on both discussed benchmarks. 3) There have been some significant advances in OV methods up until the time of writing. CLIPSeg could be replaced by the newer and more accurate SAM [18] which would most likely improve TOSIP benchmarks on cases A and B. 4) Finally, TOSIP can be transitioned from simulation to real-world deployment through Sim2Real techniques.

Figure 5.1: Google Scanned Objects

Bibliography

[1] T. Lüddecke and A. S. Ecker, "Image segmentation using text and image prompts," 2022.

[2] H. Hasselt, "Double q-learning," *Advances in neural information processing systems*, vol. 23, 2010.

[3] A. Zeng, S. Song, S. Welker, J. Lee, A. Rodriguez, and T. Funkhouser, "Learning synergies between pushing and grasping with self-supervised deep reinforcement learning," 2018.

[4] OpenAI, "Gpt-4 technical report," 2023.

[5] e. a. Kerr, Justin, "Lerf: Language embedded radiance fields." in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[6] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3d scene graph construction and optimization," 2022.

[7] S. Peng, K. Genova, C. M. Jiang, A. Tagliasacchi, M. Pollefeys, and T. Funkhouser, "Openscene: 3d scene understanding with open vocabularies," 2023.

[8] e. a. Radford, Alec, "Learning transferable visual models from natural language supervision," 2021.

[9] e. a. Jia, Chao, "Scaling up visual and vision-language representation learning with noisy text supervision," 2021.

[10] X. Chen, H. Fang, T.-Y. Lin, R. Vedantam, S. Gupta, P. Dollar, and C. L. Zitnick, "Microsoft coco captions: Data collection and evaluation server," 2015.

[11] B. A. Plummer, L. Wang, C. M. Cervantes, J. C. Caicedo, J. Hockenmaier, and S. Lazebnik, "Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models," 2016.

[12] B. Li, K. Q. Weinberger, S. Belongie, V. Koltun, and R. Ranftl, "Language-driven semantic segmentation," 2022.

[13] Y. Rao, W. Zhao, G. Chen, Y. Tang, Z. Zhu, G. Huang, J. Zhou, and J. Lu, "Denseclip: Language-guided dense prediction with context-aware prompting," 2022.

[14] C. Ma, Y. Yang, Y. Wang, Y. Zhang, and W. Xie, "Open-vocabulary semantic segmentation with frozen vision-language models," 2022.

[15] V. Dumoulin, E. Perez, N. Schucher, F. Strub, H. de Vries, A. Courville, and Y. Bengio, "Feature-wise transformations," *Distill*, 2018.

[16] G. Ghiasi, X. Gu, Y. Cui, and T.-Y. Lin, "Scaling open-vocabulary image segmentation with image-level labels," 2022.

[17] F. Liang, B. Wu, X. Dai, K. Li, Y. Zhao, H. Zhang, P. Zhang, P. Vajda, and D. Marculescu, "Open-vocabulary semantic segmentation with mask-adapted clip," 2023.

[18] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," 2023.

[19] P. K. Murali, A. Dutta, M. Gentner, E. Burdet, R. Dahiya, and M. Kaboli, "Active visuo-tactile interactive robotic perception for accurate object pose estimation in dense clutter," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4686–4693, apr 2022.

[20] H. Zhang, Y. Lu, C. Yu, D. Hsu, X. La, and N. Zheng, "Invigorate: Interactive visual grounding and grasping in clutter," 2021.

[21] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," 2023.

[22] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-Maron, M. Gimenez, Y. Sulsky, J. Kay, J. T. Springenberg, T. Eccles, J. Bruce, A. Razavi, A. Edwards, N. Heess, Y. Chen, R. Hadsell, O. Vinyals, M. Bordbar, and N. de Freitas, "A generalist agent," 2022.

[23] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," 2022.

[24] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[25] C. J. C. H. Watkins, "Learning from delayed rewards," 1989.

[26] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," 2018.

[27] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, Sendai, Japan, Sep 2004, pp. 2149–2154.

[28] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.

[29] B. Ellenberger, "Pybullet gymperium," https://github.com/benelot/pybullet-gym, 2018–2019.

[30] E. Rohmer, S. P. Singh, and M. Freese, "V-rep: A versatile and scalable robot simulation framework," in *2013 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2013, pp. 1321–1326.

[31] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[32] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2016.

[33] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, pp. 279–292, 1992.

[34] B. Calli, A. Walsman, A. Singh, S. Srinivasa, P. Abbeel, and A. M. Dollar, "Benchmarking in manipulation research: Using the yale-CMU-berkeley object and model set," *IEEE Robotics &amp Automation Magazine*, vol. 22, no. 3, pp. 36–52, sep 2015.

[35] L. Downs, A. Francis, N. Koenig, B. Kinman, R. Hickman, K. Reymann, T. B. McHugh, and V. Vanhoucke, "Google scanned objects: A high-quality dataset of 3d scanned household items," 2022.

Appendix A

TOSIP

A.1 LLM Capabilities

LLMs provide good reasoning capabilities and an intuition on where to search an environment to find a target object. As an example, we prompt a GPT-4 [4] with the following text:

I will ask for the most likely places to look for a household object. Your response should come in the form of a list, by providing a maximum of three places. For instance: input: "spoon" output: ["kitchen drawer", "dining table", "dishwasher"].

The following examples illustrate the viability of the aforementioned statement:

cup ["bedroom dresser", "living room table", "pants pocket"]

pants ["kitchen cabinet", "sink", "dining table"]

usbc charger ["desk", "nightstand", "backpack or bag"]

backpack ["bedroom floor", "closet", "entryway or mudroom"]

detergent ["laundry room shelf", "utility closet", "under the sink"]

sponge ["kitchen sink", "under the sink cabinet", "countertop"]

broom ["utility closet", "kitchen corner", "garage"]

A.2 Preprocessing

The computation of the minimal required image padding for 16 rotations is shown in eq. (A.1). Padding an image is illustrated for Fig. 3.4d with Fig. A.1. Padded images are then rotated, which is demonstrated in Fig. A.2. Depth and segmentation ground truths are preprocessed analogously.

$$H_{pad} = W_{pad} = \lceil sin(\alpha_{max}) \cdot d \rceil \cdot H = 318 \tag{A.1}$$

where: H_{pad} : padded image height
W_{pad} : padded image width
α_{max} : angle at which maximal padding is required = 45°
d : image diagonal = $H \cdot \frac{\sqrt{2}}{2}$

Figure A.1: RGB image padded

A.3 Evaluation Scenes

The following Fig. A.3 and Fig. A.4 show evaluation scenes which were used to train TOSIP-no-OV and TOSIP, respectively.

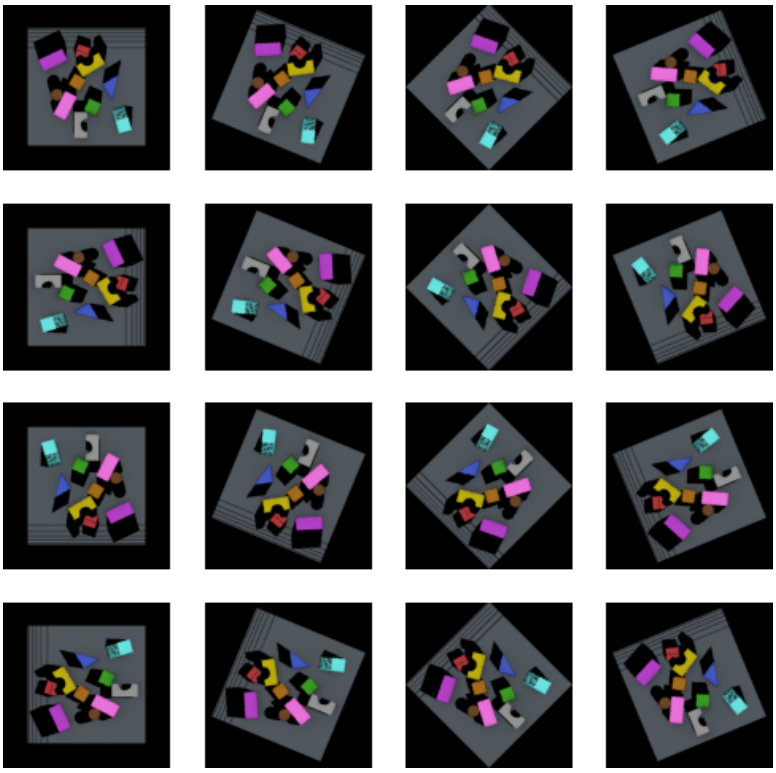


Figure A.2: Rotated RGB image

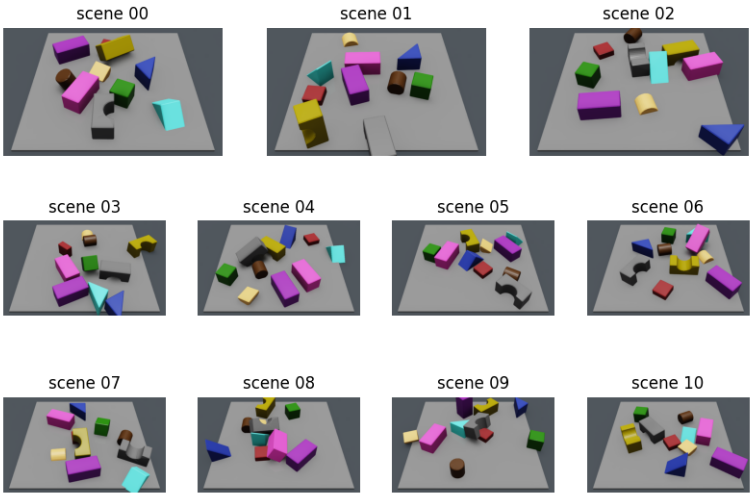


Figure A.3: TOSIP-no-ov Eval Scenes

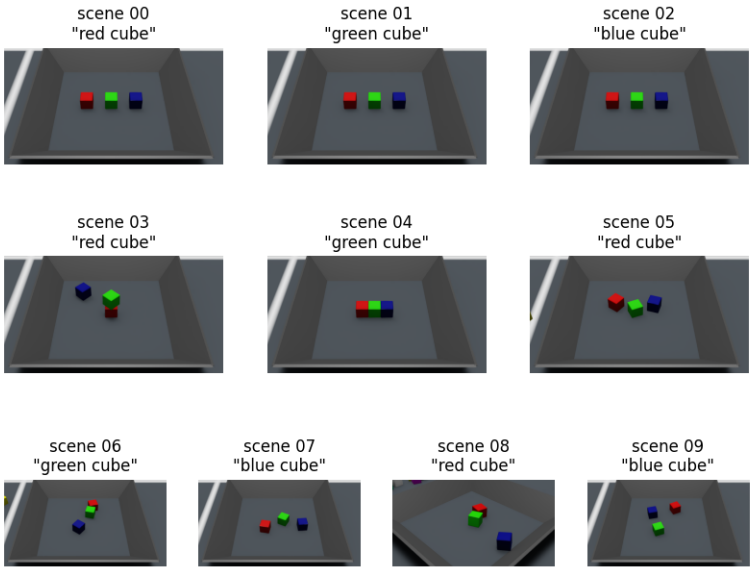


Figure A.4: TOSIP Eval Scenes (Cubes)